ASTANA IT UNIVERSITY
School of Software Engineering
Software Engineering programme

First Pre-defense Report

Development of a Reproducible Testing Platform for Microservices with Interaction Management and Failure Scenario Reduction.

Working product name: ReproService
Team, group SE-2416
Tamerlan Khassenov
Aldiyar Gaisa
Askhat Rakhmetollayev
Supervisor: Arailym Orazova, Senior-Lecturer, School of Software Engineering
First pre-defense: 24 October 2026
Report prepared: 14 October 2026
Astana, 2026

Contents

1 Project proposal 3
2 User needs validation 5
3 Literature review 6
4 Comparison of existing systems 9
5 Technology selection 11
6 Requirements and acceptance criteria 12
7 Architecture and contracts 13
8 UX mockup and usage scenarios 18
9 Experimental evaluation plan 19
10 Team plan and risks 21
11 Supervisor review 22
12 Readiness and team actions 23
13 References 25

Abstract

ReproService is a proposed local platform for reproducing microservice failures, reducing fault interventions and checking fixes. This report defines the problem and MVP scope, reviews 23 sources, compares five systems, specifies requirements and architecture, and presents an evaluation protocol.

Project documentation and an interactive mockup have been prepared. A working backend and empirical results remain to be produced. The research hypothesis concerns candidate ordering and reduction cost while preserving the original violation. Superiority over existing tools is not asserted before evaluation.

1 Project proposal

Official title: Development of a Reproducible Testing Platform for Microservices with Interaction Management and Failure Scenario Reduction.

The working product name is ReproService. It does not imply that the name is a unique trademark.

Team and supervision

The team belongs to group SE-2416, Software Engineering, Astana IT University. Tamerlan Khassenov is responsible for the proxy and event scheduling; Aldiyar Gaisa for environment reset, assertions and scenario reduction; Askhat Rakhmetollayev for the interface and visualization. The supervisor is Arailym Orazova, Senior-Lecturer, School of Software Engineering.

Problem and intended users

A service may complete an operation but lose its response. Retrying the request can charge a customer or reserve inventory twice. Changes in message ordering mean that rerunning an ordinary test may not reproduce the failure. Studies S01, S02 and S05 support the relevance of controlled failure testing. Demand for this particular product still needs to be validated through interviews.

Intended users are backend developers, test automation engineers and platform engineers. The prospective business stakeholder is the development lead of a team using HTTP microservices. Kazakhstan is the intended location for an initial pilot; the absence of local alternatives has not been established.

Aim and objectives

The aim is to develop and experimentally evaluate a local platform that records and replays supported interservice failure scenarios, reduces the number of fault interventions and helps verify fixes.

1. Derive requirements from literature, existing systems and 3–5 interviews.
2. Define the models of events, interventions, the original violation and testbed state.
3. Implement an HTTP proxy, orchestration and scenario replay.
4. Add API/SQL assertions and scenario reduction.
5. Implement an event timeline and version comparison interface.
6. Measure reproducibility, scenario size and reduction cost against baseline algorithms.

MVP scope

Included: one local Docker Compose testbed with orders, payments and inventory services; HTTP/JSON; a test PostgreSQL database; request delays, response loss after successful execution and prescribed HTTP errors; sequential experiments; JSON scenarios; event logs; predefined assertions; and a basic reducer.

Excluded: production deployment, Kubernetes, Kafka/RabbitMQ, TLS interception, complete control of threads, clocks or randomness, universal root cause analysis, and support for arbitrary applications without adaptation. Scheduling independent HTTP events is an extension after basic replay works.

Research question

Does prioritizing reduction candidates using observed event relationships reduce the number of trial executions while preserving the original violation, compared with ordinary ddmin under the same budget?

The main hypothesis is a lower execution cost without materially degrading reproducibility. The null hypothesis is that no consistent improvement is observed. A negative result is acceptable and must be reported.

Expected outcome and current status

The intended final deliverables are a working platform, reproducible testbeds, known faulty and fixed versions, measurement logs, documentation and comparative evaluation. As of 14 October, project materials and a standalone interactive UX mockup have been prepared. The proxy, Docker orchestration and empirical evaluation have not yet been implemented. The mockup uses synthetic data.

Practical value

The expected benefit is less effort in preparing repeatable experiments and checking fixes. Business value must be established through interviews and a pilot. A possible commercial model is deployment and support for development teams; pricing and willingness to pay have not been established.

An additional experiment compares a failure before an operation with response loss after successful execution. Its protocol is specified in Section 9 and tests/POST_COMMIT_EXPERIMENT.md. The official project title remains unchanged.

2 User needs validation

Interviews have not yet been conducted. Literature establishes the existence of the problem class, but does not demonstrate demand for ReproService. The plan is to hold 3–5 discussions with backend, QA or platform engineers from different teams by 18 October. At least two participants should have personally investigated interservice failures.

Invitation and consent

Suggested invitation: “We are AITU students researching how HTTP microservice failures are reproduced. Could we discuss your experience for 20 minutes? We do not need source code, customer personal data or trade secrets. With your permission, we would retain anonymized notes for our diploma project. Recording the conversation requires separate consent.”

Interview questions

1. Describe the most recent interservice failure you investigated. What happened and how was it detected?
2. How did you try to reproduce it? Which approaches did not work?
3. How long did the investigation take? Please indicate whether this is an estimate.
4. Which tools did you use, and what specific capability was missing?
5. What prevents you from reproducing the issue on a local testbed?
6. Which assertions distinguish the original defect from an unrelated failure?
7. How do you verify that a fix works?
8. What restrictions apply to proxies, Docker or data transfer in your environment?
9. Who decides whether to adopt such a tool, and what would a pilot require?
10. After discussing the problem, examine the mockup. Which steps are unclear? Which outputs would be useful or unnecessary?

Asking whether someone likes the idea is not evidence of demand. Separate accounts of actual incidents, feature requests and the team's interpretations.

Evidence record

Each record should contain an INT-ID, date, role and experience, system type, consent, anonymized quotation, incident description, existing tool, constraint, related FR/NFR requirement, and a distinction between fact and interpretation. Keep contact details privately. Do not upload contacts or interview recordings to Git. Obtain permission for follow-up validation.

Decision rule

Group observations by reproducibility, investigation effort and adoption constraints. Link each scope change to an INT-ID. If only one team confirms the need, describe it as a pilot case. If no participant is willing to install a proxy, reconsider integration before implementation. Lack of demand is also a valid finding.

The INT-01–INT-05 register remains open. No interview evidence has been entered.

3 Literature review

Search date: 14 October 2026. Search terms combined microservices, fault injection, reproducibility, delta debugging and failure minimization. Inclusion criteria were direct relevance to failure discovery, replay or reduction, an author or publisher source, and a verifiable bibliographic record. Marketing roundups, duplicates and unrelated physical attack papers were excluded. This is a targeted critical review, not an exhaustive systematic review.

The scientific core contains nine publications, S01–S09. Eight are dated 2021–2026. S09, published in 2016, is retained as foundational work. S03 uses its 2022 journal issue year, while its DOI reflects early publication in 2019. S08 was verified as an ICSE Companion 2024 paper; the link points to an author version. D01–D05 are product sources and do not count as scientific publications. T01–T09 support technical and organizational decisions. There are 23 sources in total.

Analytical comparison

Source	Approach and finding	Limitation for this project	Application
S01 · 2026	Network fault injection and a dynamic behavior model reduce the combinations explored.	Searching combinations does not establish the effectiveness of a reducer for an already discovered scenario. Request identification assumptions matter for replay.	Closest technical reference; compare within a shared HTTP scope and define event identity.
S02 · 2021	Filibuster generates partial failure variants of functional tests and reduces the search space.	Search space reduction and minimization of a particular counterexample are different tasks. Fault injection itself is established.	Reuse correctness assertions and distinguish failure discovery from subsequent reduction.
S03 · 2022	Delta debugging reduces changes to the microservice environment; infrastructure automates deployment and parallel testing.	Its unit of reduction differs from an HTTP intervention. Parallel execution requires isolation.	Define the reduction unit explicitly and run candidates sequentially in the MVP.
S04 · 2024	FaultFuzz uses coverage and I/O feedback to prioritize fault combinations.	System coverage requires instrumentation; results do not transfer directly to an HTTP proxy.	Start with a limited budget and simple baselines; consider coverage later.
S05 · 2024	Anduril combines static causal analysis and feedback to reproduce a particular failure.	Reproducing a known failure differs from discovering a new one. Static analysis is outside the MVP.	Preserve the original invariant ID; an unrelated failure is not successful replay.
S06 · 2024	Legolas chooses fault injection points using abstract states inferred from code.	Internal state control requires instrumentation. HTTP observations provide less visibility.	Limit claims to observed interservice events.
S07 · 2025	CAFault considers failure-related configurations when reducing the testing space.	One configuration does not represent all operating modes. Exhaustive configuration search is outside scope.	Record timeouts, retries, image versions and seed checksums.

Source	Approach and finding	Limitation for this project	Application
S08 · 2024	Filibuster extensions inject SQL/NoSQL client faults and visualize results for developers.	Visualization and database support are not inherently novel.	Distinguish database fault injection from database queries used as business oracles.
S09 · 2016	DEMi minimizes faulty distributed executions.	The older paper is retained because distributed execution minimization is established prior work.	Evaluate size and execution cost without claiming a global minimum.
D01	Reynard documents proxies, tracing, JUnit 5 and HTTP/gRPC integration.	Documentation does not replace practical verification of the current implementation.	Inform event contracts and system comparison.
D02	Filibuster generates failure variants from functional tests.	Search space reduction is not necessarily counterexample minimization.	Compare testing and replay workflows.
D03	Antithesis describes a deterministic hypervisor, fault injection and causal analysis.	No product deployment was tested; on-premises terms are not confirmed by the reviewed page.	Distinguish full determinism from HTTP replay.
D04	Chaos Mesh supports Kubernetes failure experiments.	Repeating experiment configuration does not guarantee the same event order.	Compare orchestration and deployment requirements.
D05	Keploy records and replays API traffic and dependencies locally.	Record/replay does not imply control of all process execution.	Compare replay and fix verification workflows.

Synthesis and research direction

The literature addresses three related tasks: discovering failure combinations, reproducing a known violation, and reducing a faulty execution. Reducing a search space must not be treated as equivalent to minimizing a specific failure scenario.

The proposed platform combines HTTP event control, state restoration and scenario reduction. Each component has precedents. The research focus is whether candidate ordering based on observed relationships changes reduction cost while preserving reproducibility. Neither superiority over competitors nor uniqueness of the combination has been established.

Ordinary ddmin and a relationship-prioritized variant will use identical intervention sets and the same scenario validation. Observed relationships may guide candidate order, but do not prove that a candidate is impossible. Iterative single-intervention deletion provides an additional baseline. Section 7 specifies the revised dependency rules.

Review limitations

Source access depth is recorded in the bibliography and references.json. Some papers were reviewed only through abstracts and bibliographic records; their detailed algorithms and numerical findings must not be presented as verified. The team should study S03 and S09 in depth before implementing the reducer, and reproduce the official Reynard example before a practical comparison. Results from other studies are not measurements of ReproService.

Findings from full-text checks

The following checks concern selected sections of S01, S02 and S09. They do not certify that individual team members have personally completed the reading.

S01, Sections 3.1–3.5, 4 and 5.4: in the described model, a proxy synthesizes an error without forwarding the original request upstream. Request identity includes destination, interface, payload, predecessors and a counter, and relies on service behavior assumptions. Dependency handling and pruning impossible combinations already exist. Retry reduction has soundness limitations and is disabled by default. The project therefore distinguishes pre-execution errors from losing a response after successful execution. This is a planned distinction from the model in Section 3.1, not proof that every version of Reynard lacks the feature.

S02, Sections 3.1.2–3.1.4 and 4, Algorithm 1: replaying an individual discovered counterexample is explicitly described. Dynamic reduction concerns the space of tests, rather than necessarily minimizing interventions in one counterexample. Assumptions include deterministic responses and restrictions on dependencies on previous errors. Replay must therefore be removed from any list of unique capabilities. Business invariants, rather than endpoint availability alone, should determine correctness.

S09, Sections 2.3, 3.2.1 and 4.1–4.2: DEMi combines reduction of external events with exploration of internal event schedules. Message matching accommodates changing fields, and execution control requires instrumentation. The paper distinguishes 1-minimality from a global minimum and explicitly limits applicability. Causal information and event ordering are not new ideas. A single observed order does not establish a mandatory dependency.

The defensible engineering contribution is a local tool for a specified class of HTTP failures and a reproducible evaluation protocol. The research contribution remains a hypothesis to test through reduction comparisons and the pre-operation versus post-operation response-loss experiment. S03 remains assessed at abstract and metadata level.

4 Comparison of existing systems

Documentation snapshot: 14 October 2026. The five systems are assessed against the same ten criteria. NC means not confirmed by the reviewed materials; it does not mean absent. No practical deployment of these products has been completed. The ReproService column describes the intended MVP, not implemented capabilities. Sources: S01, S02, S08 and D01–D05.

Criterion	**Reynard**	**Filibuster**	**Antithesis**
1 Main purpose	Discover partial failure combinations	Failure variants of functional tests	Explore whole-system behavior
2 Control level	Network proxies	Service and client calls	Deterministic hypervisor
3 Deployment	Local or test environment	Development and CI	Antithesis environment; on-premises terms NC
4 Integration	Proxies, trace context, JUnit	Instrumentation and tests	Environment and properties
5 Fault injection	Before forwarding upstream in S01 Section 3.1	Remote call errors	Fault injection
6 Full determinism	No general guarantee	NC	Claimed by the vendor
7 Replay	Repeatable tests	Counterexample replay in S02 Section 3.1.4	Deterministic replay
8 Failure reduction	Separate reducer NC	Search space reduction; reducer NC	Causal analysis; comparable reducer format NC
9 Correctness check	Test assertions	Test assertions	Invariants and properties
10 Output	Tests and traces; optional Jaeger	Test results and IDE visualization	Traces and causal analysis
Criterion	**Chaos Mesh**	**Keploy**	**ReproService MVP**
1 Main purpose	Chaos experiments	Record and replay APIs	Replay and reduce one failure
2 Control level	Infrastructure and network	Traffic and dependencies	Selected HTTP events
3 Deployment	Kubernetes	Local and CI	Docker Compose
4 Integration	Cluster installation	CLI and traffic capture	Proxy, identifiers and seed data
5 Fault injection	Network and infrastructure faults	Not the main workflow reviewed	Before calls and response loss after upstream success
6 Full determinism	NC	NC	Not claimed
7 Replay	Repeat configuration	API record/replay	Repeat interventions and initial state
8 Failure reduction	Automatic reducer NC	Automatic reducer NC	Budgeted intervention reduction
9 Correctness check	Experiment checks	Response comparison and assertions	API/SQL invariants
10 Output	Dashboard	Test reports	Timeline, scenario and version comparison

Implications

Reynard and Filibuster are the closest references. Local deployment, proxies, Docker and tests are not advantages by themselves. Antithesis provides substantially deeper control over execution. The NC entries cannot support a claim that no competitor has a reducer.

The product hypothesis is that a unified local workflow for retaining, reducing and rechecking one business failure is useful. Algorithmic evaluation will compare methods on the same

testbed. Any practical Reynard comparison must use an overlapping supported scenario class. Claims that the platform is faster, cheaper or more accurate require empirical evidence.

The specific planned distinction from the model in S01 Section 3.1 is intervention after successful upstream execution. The PRE/POST experiment tests this distinction; the current implementation of the competitor has not been practically verified.

5 Technology selection

These choices are provisional and require supervisor review and a small technical pilot. The comparison is qualitative; performance has not been measured. Sources T01–T08 and D04 support the selection.

Decision	Alternative 1	Alternative 2	Selection and rationale	Reconsider when
Proxy and controller	Go: HTTP library and concurrency; pilot needed	Python asyncio: rapid prototyping and a familiar stack; different concurrency model	Go for the target MVP, combining HTTP interception and scheduling in one process; no speed advantage is asserted	The pilot cannot provide the necessary response control
Metadata	PostgreSQL: server, transactions, multiple clients	SQLite: embedded storage and simple local setup	PostgreSQL metadata, separated from the SUT test database	Setup cost prevents a pilot; consider SQLite for metadata
Environment	Docker Compose: a local multi-container environment	Kubernetes: cluster orchestration and more setup	Docker Compose fits one local testbed; Kubernetes is outside scope	Replay works and users demonstrate a need for cluster support
Interface	React and TypeScript: components and static typing	Vue and TypeScript: an alternative component model; Tamerlan has Vue experience	React and TypeScript provisionally selected; confirm with Askhat and Aldiyar	The team can maintain Vue more effectively before UI implementation

ADR 001 HTTP-level control

The proxy must represent response loss after successful execution while retaining observability. It completes the upstream request, records the event and then applies the fault action downstream. This tests application-level HTTP failures; OS threads and internal timers are not controlled.

ADR 002 Scenario identity

A scenario stores its schema version, container digests, seed data, timeout and retry settings, selectors, invariant and budget. Event identity uses the logical operation, service, method, path and attempt number. A random trace ID links a trace but is not a stable replay key. An unmatched selector produces INAPPLICABLE.

ADR 003 Reduction

The removable unit is an intervention, not an arbitrary request. Candidates preserve the workload of the main business operation. Data is reset before every execution. The original violation, another failure, timeout and infrastructure error remain distinct. Ordinary ddmin and iterative deletion are baselines; prioritization using observed relationships is experimental. Mandatory validation is identical for all methods.

ADR 004 Storage and access

The management API listens on 127.0.0.1. The proxy and SUT services use an isolated Compose network. Authorization and cookie headers are excluded from logs. The interface cannot attach arbitrary Docker sockets or launch arbitrary environments; only predefined fixtures are allowed. The MVP assumes trusted local users and does not implement multi-user access.

6 Requirements and acceptance criteria

Version 0.1 contains proposed requirements pending interviews and supervisor review. Must denotes mandatory MVP scope, Should denotes the next priority, and Could denotes an extension. Numerical thresholds are targets, not results. TC-<ID> uses the corresponding acceptance criterion and records conditions and evidence.

ID	Priority	Requirement	Acceptance criterion	Owner
FR01	Must	Import scenario	Accept valid JSON; reject unknown actions and missing selectors before execution.	Tamerlan
FR02	Must	Reset testbed	Restore seed and isolated database before each candidate; do not start if reset fails.	Aldiyar
FR03	Must	Match HTTP event	operation/service/method/path/attempt identifies exactly one event; zero or multiple matches yield INAPPLICABLE.	Tamerlan
FR04	Must	Inject delay and HTTP error	Apply the configured delay/status; synthesize an error before forwarding upstream and log the action.	Tamerlan
FR05	Must	Lose successful response	Complete the upstream operation, then close downstream without sending the response; log both events.	Tamerlan
FR06	Must	Check invariant	Return the invariant ID and expected/actual values from an API/SQL assertion; an infrastructure error is not a violation.	Aldiyar
FR07	Must	Replay scenario	Store digest, seed, timeouts and selector; block environment mismatches until a new version is explicitly created.	Tamerlan
FR08	Must	Reduce interventions	ddmin, iterative deletion and prioritization use the same scenario, validation and budget; preserve the original invariant ID.	Aldiyar
FR09	Must	Cancel execution	Stop scheduling candidates, release resources and record CANCELLED rather than PASS.	Aldiyar
FR10	Must	Show history and timeline	Filter by service and event type; show the selected event's selector and intervention outcome.	Askhat
FR11	Must	Compare versions	Display both digests, the same seed/scenario and separate outcomes; do not declare a fix verified without repetitions.	Askhat
FR12	Must	Export results	JSON includes schema_version, configuration, outcome and event references; secret headers are absent.	Askhat
FR13	Should	Control request ordering	Dependencies form a DAG; reject cycles; release an event only after its predecessor.	Tamerlan
FR14	Could	Run from CI	CLI returns distinct exit codes for PASS, invariant violation and infrastructure error.	Aldiyar
NFR01	Must	Isolation	Bind API to localhost; isolate the test network; do not execute arbitrary Compose or SQL input.	Tamerlan
NFR02	Must	Confidentiality	Mask Authorization, Cookie and Set-Cookie; a seeded secret does not appear in JSON or logs.	Tamerlan
NFR03	Must	Reproducibility	Target: at least 27 of 30 replays reproduce the original invariant ID; report the rate and confidence interval.	Aldiyar
NFR04	Should	Proxy overhead	Target: added p95 latency without faults is at most 20 ms at 100 requests/s on the specified testbed; record hardware.	Tamerlan
NFR05	Must	Usability	Target: 4 of 5 representative users open a scenario, identify the violation and export results within 5 minutes without hints.	Askhat
NFR06	Must	Resource budget	Stop at 100 executions, including confirmations, or 10 minutes, whichever occurs first; record BUDGET_EXHAUSTED.	Aldiyar

7 Architecture and contracts

The diagrams describe the proposed architecture, not a functioning backend. Editable originals and exported diagrams are included in the repository.

System boundary and components

An engineer uses a browser to control the platform. The controller launches only an allowlisted Docker Compose testbed. HTTP proxies sit between orders, payments and inventory. Platform metadata is separate from the SUT PostgreSQL database. Faults are not directed at production systems.

- Web UI: scenario configuration, execution status, timeline, comparison and export.
- Controller API: validation, serialized runs, lifecycle and cancellation; localhost only.
- Proxy and scheduler: event matching, fault interventions and observed relationship logging.
- Runner and reset: image digest checks, seed restoration, health checks, workload execution and cleanup.
- Oracle: allowlisted API/SQL assertions returning a specific invariant ID.
- Reducer: ordinary ddmin, iterative deletion and a relationship-prioritized variant using the same runner and oracle.
- Metadata store: scenario, version, run, event, candidate and assertion records, with append-only events and final status.

Execution lifecycle

VALIDATING → RESETTING → READY → RUNNING → ASSERTING → COMPLETED. Active states may transition to CANCELLED or INFRA_ERROR. A completed run separately records PASS, ORIGINAL_FAILURE, OTHER_FAILURE, TIMEOUT or INAPPLICABLE. TIMEOUT is not the original failure unless it matches a predefined timing invariant with the same ID.

Reset removes only the test volume owned by that run; unrelated containers are untouched. The seed hash and health checks are verified before execution. Users select predefined fixture and assertion IDs rather than arbitrary shell commands or SQL.

Event model and stable matching

Event fields: event_id, run_id, logical_operation, source, target, method, normalized_path, attempt, parent_event_id, monotonic_offset_ms, action_id, phase and result. traceparent and tracestate [T06] link events, but random trace IDs are not replay matching keys.

The selector combines logical_operation, source, target, method, normalized_path and attempt. For the MVP, the test driver assigns deterministic attempt numbers. Ambiguous concurrent matches are unsupported. If no event matches within the time window, the result is INAPPLICABLE. Sensitive headers are removed; request bodies are not retained by default.

Reduction procedure

1. Confirm the original violation on the full intervention set.

2. Partition interventions and test removals using ordinary ddmin.

3. Apply explicit scenario preconditions equally to every algorithm. In the experimental variant, observed relationships guide candidate order, not unconditional pruning.

4. Reset the testbed, execute the candidate and evaluate the oracle. Accept only ORIGINAL_FAILURE with the same invariant ID, confirmed in three executions out of three.

5. Stop when no further deletion is accepted, the budget is exhausted or cancellation is requested. Store the best confirmed candidate and the stopping reason.

Confirmation runs consume the shared budget. An unstable candidate is rejected and labeled unstable. The full initial scenario is checked under the same rules for every algorithm. Interventions are removed, not business requests. A global minimum is not claimed. A 1-minimal result may be claimed only after every single deletion has been checked and fails under the stated conditions.

Proposed API

POST /api/scenarios validates and returns a version ID. POST /api/runs returns 202 and a run ID. GET /api/runs/{id} returns status. GET /api/runs/{id}/events?cursor= retrieves events. POST /api/runs/{id}/cancel cancels execution. POST /api/runs/{id}/reduce starts reduction. GET /api/runs/{id}/export exports a result.

Run creation uses an idempotency key: repeating a POST must not create a second run. A busy testbed returns 409 for another run. JSON uses schema_version=1. Errors include code, message, field and correlation_id.

Dependency rules under ADR 005

Status: proposed following literature review; pending team and supervisor review.

The initial use of “dependency” was too broad. Observing B after A does not prove that B is impossible without A. Removing an intervention can expose a different execution path. Pruning a candidate on that basis may hide a smaller reproducible failure.

Explicit preconditions are part of the scenario contract: B is permitted only after A. The validator checks them identically for all algorithms. Observed relationships come from a particular trace and may prioritize candidates, but do not establish impossibility.

The experimental variant changes the order in which blocks are tested for removal while retaining the same validation as ddmin. Ties use stable ID order. A proposed heuristic first tests blocks weakly connected to events associated with the business invariant. Graph proximity is not proof of causality. Fix the scoring formula before the main experiment; tuning scenarios must not be included in the evaluation set.

Iterative deletion removes one intervention at a time and restarts the pass after an accepted removal. This baseline establishes whether added complexity is justified. All algorithms share the runner, oracle, cache and confirmation budget.

Correctness checks cover B occurring after A but also being possible without A; an explicit B-requires-A contract; a candidate causing a different invariant; equal graph scores; and a budget exhausted during confirmation. In the last case the candidate is not confirmed. The research concerns ordering cost, not an unfair comparison against a baseline lacking required validation. A null or negative effect remains valid.

The platform provides no full determinism of OS execution, threads or clocks. Results on the demonstration testbed do not establish applicability to arbitrary distributed systems.

System context

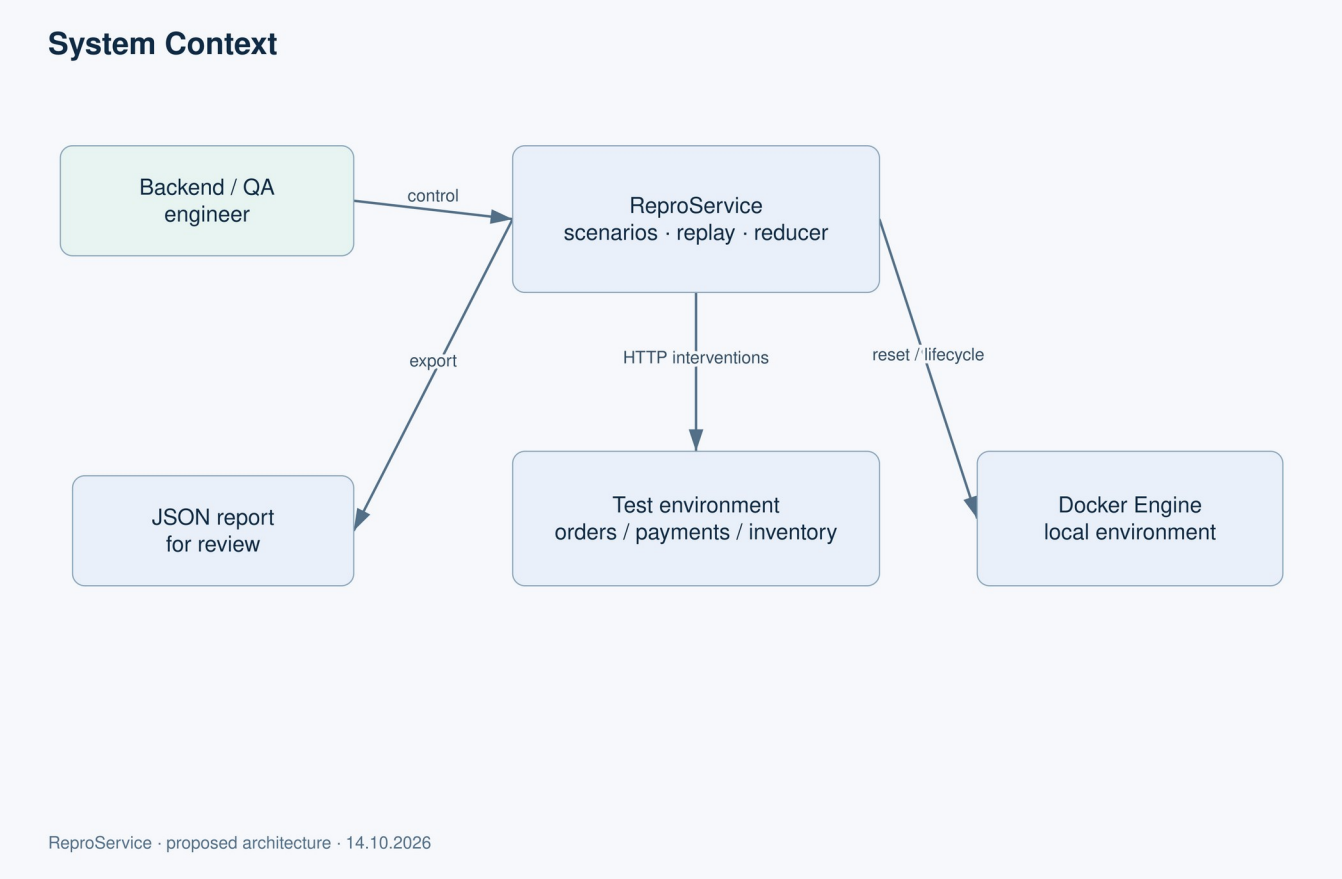


Figure 1 — System context

Proposed architecture. Editable English diagrams are provided in diagrams/en in the repository.

Use cases

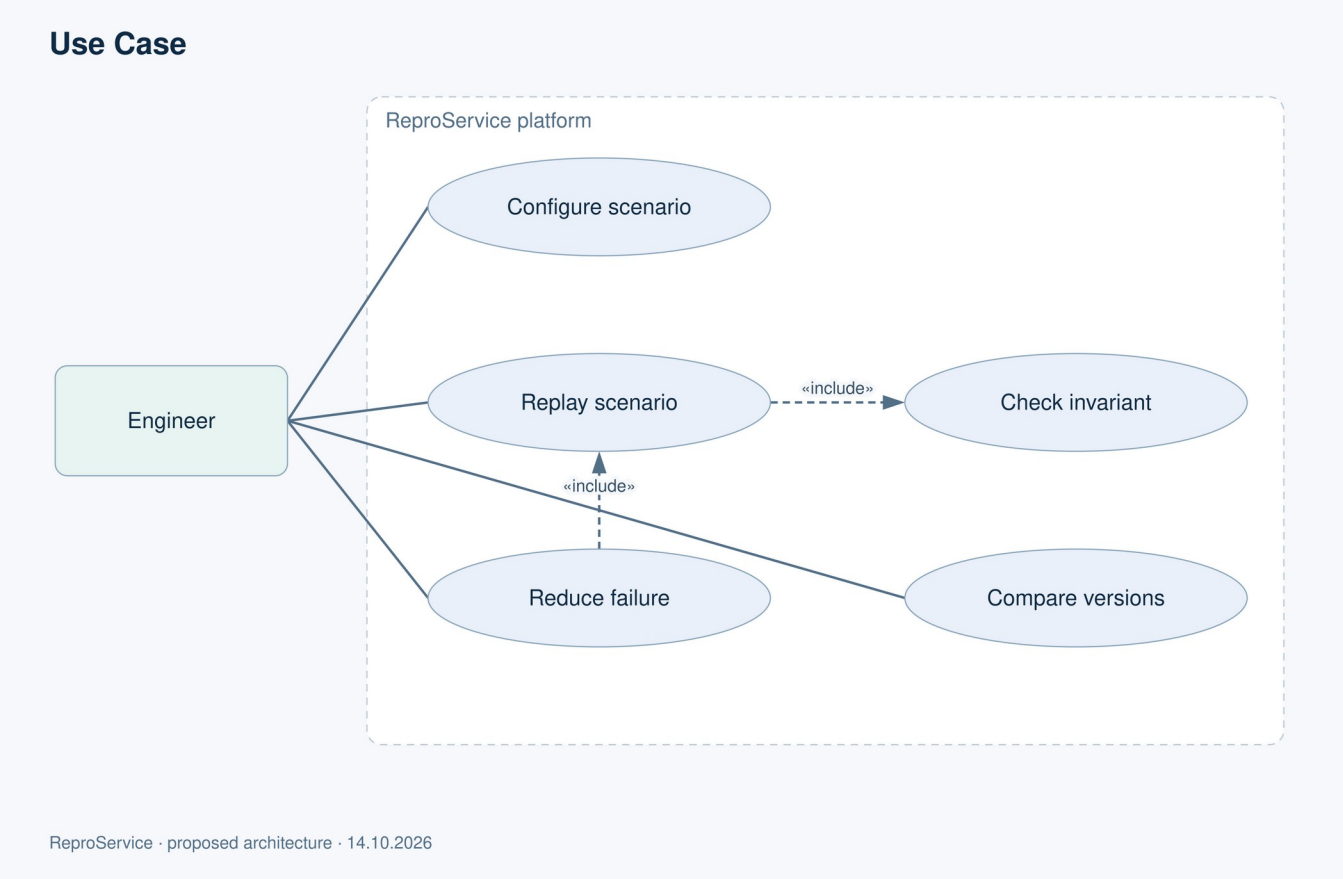


Figure 2 — Use cases

Proposed architecture. Editable English diagrams are provided in diagrams/en in the repository.

Platform components

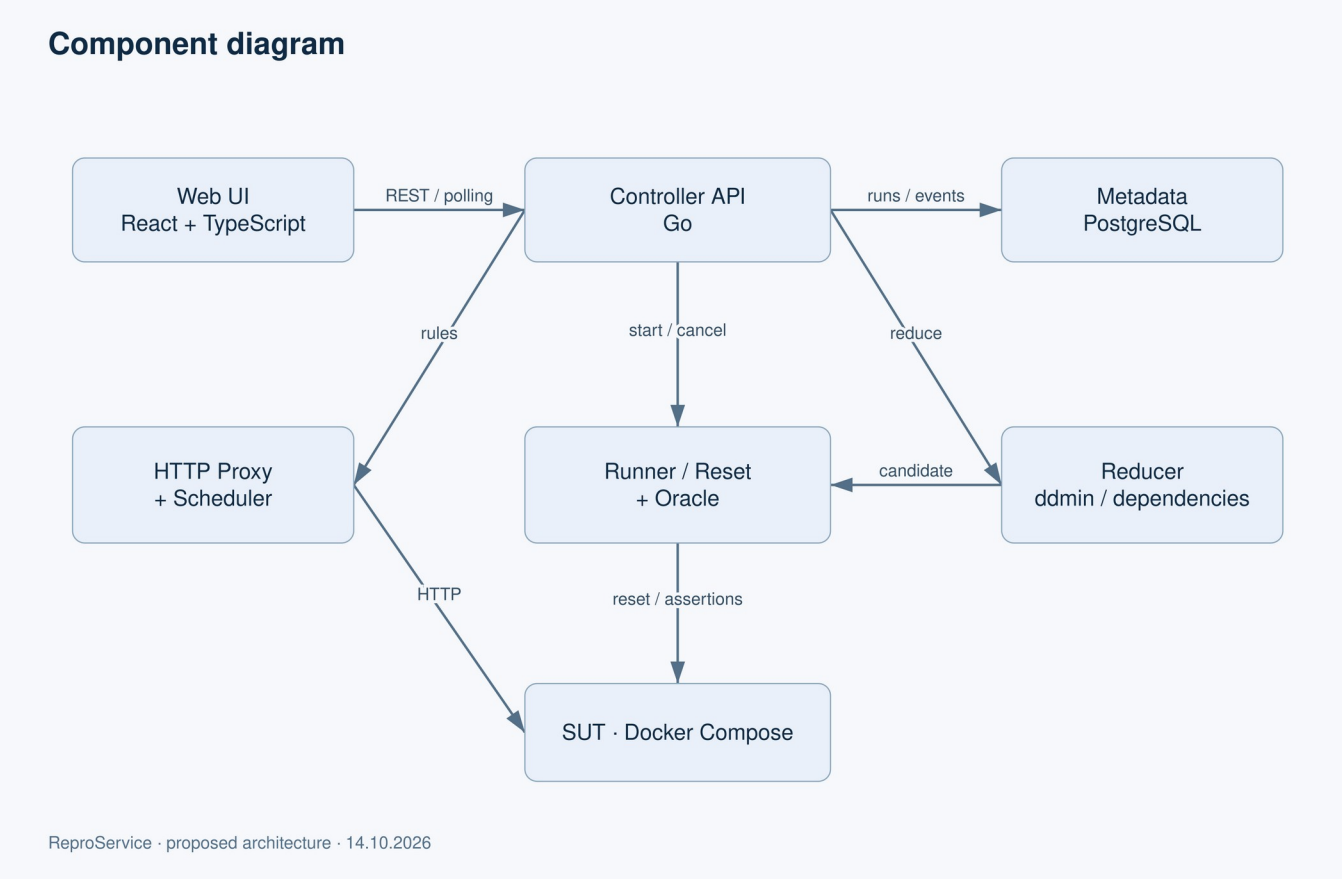


Figure 3 — Platform components

Proposed architecture. Editable English diagrams are provided in diagrams/en in the repository.

8 UX mockup and usage scenarios

Open prototype/index.html in a browser without installation. Its events and values are synthetic and it is not connected to a backend. The existing interactive mockup is supplied separately in the repository.

Step	Engineer action	Displayed result
1	Select a fault intervention and run the mockup	Run status and HTTP events
2	Open an event on the timeline	Selector, intervention and invariant ID
3	Start reduction	Original and reduced intervention sets
4	Compare the fixed version	Separate outcomes for both versions
5	Export	Demonstration JSON

Reset returns the mockup to its initial state. Later actions are unavailable before the initial run.

For usability validation, ask a participant to investigate duplicate charging, reduce the scenario and export a result without explaining the controls. Record completion time, mistakes, unclear wording and requests for help. Check keyboard use, contrast, text wrapping and a 390 px viewport. Five real participants and their results are not yet available.

The intended product states include an empty list, validation, execution, cancellation, the original violation, another failure, infrastructure error, an inapplicable scenario and an exhausted budget. The mockup illustrates the main path rather than implementing every product state.

9 Experimental evaluation plan

This protocol precedes implementation. The results.csv file contains no measurements. Synthetic UX values must not be used as experimental results.

Research question and hypotheses

Does ordering candidates using observed relationships reduce failure scenario reduction cost while preserving the same violation, compared with ordinary ddmin?

H1: relationship-based prioritization reduces the number of executions needed to obtain a confirmed result. H2: the final reproduction rate is no more than 10 percentage points below the baseline. These are hypotheses, not established advantages.

Scenario set

Six scenarios are planned: response loss after payment; repeated inventory reservation after timeout; inventory failure after successful payment; delay exceeding retry timeout; two independent failures; and a dependent chain of three interventions. Each requires faulty and fixed versions, an invariant ID and a fixed seed. Include scenarios that are not designed to favor the proposed method. A scenario that does not expose a failure is reported as a negative result.

Comparison conditions

All algorithms use the same image digests, seed, CPU/RAM limits, workload, oracle, full initial candidate, operation ordering and budget of 100 executions or 10 minutes. Reset precedes each candidate. Three confirmation executions count toward the budget. Record both execution count and elapsed time.

Alternate baseline and variant order. Perform 10 independent reduction trials per scenario for each of ordinary ddmin, iterative deletion and the proposed variant. Record OS, software versions and background processes. The scenario is the unit of analysis; nested executions are not independent observations.

Metrics

- Reproducibility: ORIGINAL_FAILURE occurrences out of 30 replays of the best scenario, with a 95% Wilson confidence interval. Report INFRA_ERROR and TIMEOUT separately rather than silently excluding them.
- Size: interventions before and after reduction, and fraction removed. Do not label the result a global minimum.
- Cost: all executions, confirmations, rejected invalid candidates, reset time and total elapsed time.
- Budget: frequency of budget exhaustion. Do not replace incomplete outcomes with zero.
- Proxy performance: direct requests versus a proxy without injected faults, using the same 100 requests/s workload for 60 seconds after warm-up; record p50, p95, p99 and errors.

For each scenario, present the median and range over 10 trials. Report paired differences and bootstrap confidence intervals where the number of scenarios supports them. With only six scenarios, conclusions are preliminary and generalization is limited. Include cases where baselines perform better.

Functional tests and threats to validity

TC-FR01–14 and TC-NFR01–06 follow the requirements in Section 6. Negative cases include ambiguous selectors, dependency cycles, failed reset, a different invariant, a missing

container, cancellation, exhausted budgets, modified seed data and secret headers. None should be incorrectly counted as successful replay.

Threats include a small testbed, deliberately introduced defects, machine variability, subjective dependency selection and oracle errors. Mitigations are a predefined scenario set, versioned protocols, raw logs, independent oracle review and at least one external pilot. Results from other publications must remain separate from our measurements. Scenario validation and cache rules are identical across all reduction methods.

Experiment on post-operation response loss

The experiment is designed but not yet executed. It tests which duplicate-side-effect defects are exposed by losing a response after upstream completion compared with injecting an error before the call. The comparison is grounded in the model described in S01 Section 3.1, not a claim about all chaos tools.

The orders service calls payments POST /charge and retries once after a network error. BUG does not enforce idempotency. FIX retains and reuses an idempotency key, and payments guarantees one charge per key. Reset restores an empty payment ledger. Inventory remains available throughout this experiment.

Mode	Intervention timing	Expected check rather than measured result
CONTROL	No fault	One successful charge
PRE	Error before the first request is forwarded to payments	First attempt does not change the ledger; check the effect of retry
POST	payments completes its write, then the first response is lost	Check whether retry creates a second charge

Use the same checkout, amount, retry policy, reset and oracle in every mode. Compare BUG and FIX in all three modes, with 30 runs per cell. Do not transfer mockup numbers into results.

The oracle checks confirmed charges per checkout_id, operation-key uniqueness and the final outcome. ORIGINAL_FAILURE means more than one charge for the original invariant. Zero charges after a failed checkout is a different outcome. For FIX, HTTP 200 alone is insufficient; the ledger must also pass validation.

Record upstream commit → proxy drop → retry ordering. Without evidence of commit, classify a POST run as invalid. A simple HTTP 503 before upstream execution does not model POST.

If POST exposes BUG while PRE does not, the result supports the advantage of that fault model for that scenario class. It does not establish superiority of the entire platform over Reynard or other products. If both models detect the defect, report the absence of the hypothesized distinction.

10 Team plan and risks

This is a coordination plan, not evidence of work already completed by individual students. Each owner must personally review and be able to explain the materials prepared with assistant support.

Period	Deliverable	Owner	Dependency
14–15 October	Reviewed scope, title, roles, repository and tracker	Tamerlan	Team access
15–18 October	3–5 interviews, paper reading and updated requirements	Whole team	Participant consent
18–20 October	Architecture, oracle, experiment protocol and UX review	Aldiyar, Askhat and Tamerlan	Requirements
21 October	Supervisor discussion	Whole team	Actual meeting
22 October	Address feedback and conduct cross-review	Non-author reviewers	Supervisor comments
23 October	Rehearsal and link/export checks	Askhat	Updated materials
24 October	Agreed package and pre-defense-1 tag	Tamerlan	Completed review

After the first pre-defense, the initial effort estimate is two weeks for the testbed/proxy, two for oracle/replay, two for reducer/UI, and two for experiments and reporting. Later defense dates must be checked against the university calendar.

Ownership and peer review

Tamerlan owns architecture, proxy, scheduling and integration; Aldiyar reviews this work. Aldiyar owns reset, assertions, reducers and evaluation; Tamerlan reviews it. Askhat owns the interface, timeline, accessibility and presentation; Tamerlan reviews it, with Aldiyar joining UX review. All members participate in interviews and literature reading. Prepared drafts are not evidence of personal implementation.

Risk	Likelihood and impact	Early indicator	Mitigation and owner
Unstable replay	High / high	Fewer than 27 successful replays out of 30	Reduce concurrency and fix the seed; Tamerlan
Incorrect oracle	Medium / high	OTHER_FAILURE accepted as original	Independent assertion review; Aldiyar
Existing tool covers the contribution	High / medium	Matching method in S01, S03 or S09	Refine the experimental contribution; whole team
Interviews delayed	Medium / high	Fewer than three agreements by 16 October	Contact more engineers and report the evidence gap; Askhat
Excessive scope	High / high	Must requirements remain incomplete	Drop FR13/FR14 and retain sequential HTTP testing; Tamerlan
Reset or containers hang	Medium / medium	Time limits exceeded	Timeout and run-owned cleanup; Aldiyar
Sensitive logs exposed	Low / high	Body or token appears in export	Field allowlist and masking tests; Tamerlan

11 Supervisor review

Arailym Orazova, Senior-Lecturer, School of Software Engineering, Astana IT University. Contact: arailym.orazova@astanait.edu.kz.

Feedback and approval have not yet been received. This report is not an approval statement by the supervisor.

Proposed meeting agenda

1. Review the exact title in three languages and MVP scope — 5 minutes.
2. Discuss the contribution relative to Reynard, Filibuster and DEMi — 7 minutes.
3. Review the research question, baselines and original-violation criteria — 8 minutes.
4. Agree on individual contributions, deadlines and acceptance conditions — 5 minutes.
5. Record comments and the next checkpoint — 5 minutes.

The proposed supervision schedule is a weekly 20-minute meeting, with PR links and questions submitted one day in advance. Intermediate demonstrations follow proxy, replay and reducer milestones. Support includes joint investigation of unstable scenarios and scope adjustment when delivery is at risk.

Date	Supervisor comment	Team response	Owner	Change reference	Confirmed
Pending	Awaiting the actual discussion	Pending	Pending	Pending	No

After the meeting, attach a record of comments approved for disclosure. Do not fabricate a signature, date or consent.

12 Readiness and team actions

Requirement	Prepared material	Remaining work
Project proposal	Proposal and scope	Personal review and approval
Practical relevance	Interview guide	Conduct 3–5 real interviews
At least 12 sources and 8 scientific papers	23 entries, including 9 scientific papers	Personal reading and deeper full-text review where access was limited
3–5 alternatives and 6–10 criteria	Five systems and ten criteria	Practically investigate disputed NC entries
At least three technology decisions	Four decisions with two alternatives each	Review ADRs
Requirements	FR01–14 and NFR01–06	Refine using interviews
Context, use case and component diagrams	Editable diagrams and exports	Review boundaries and relationships
UX mockup	prototype/index.html	Test with real users
Evaluation plan	Main and PRE/POST protocols	Implementation and measurements at a later stage
Git and YouTrack	Structure, backlog and instructions	Create remote projects and authentic activity history
Supervisor feedback	Agenda and comment register	Receive feedback and address it
Presentation	presentation/Predefense_1.pptx	Rehearse and update after review

The preparation package is available, but external evidence requirements remain open. Do not create the final submission tag before review is complete.

Repository and task tracker setup

No remote repository or YouTrack server is connected. PLAN-xx identifiers in backlog.json are local planning numbers, not actual tracker issue IDs.

1. Create a private GitHub or GitLab repository and invite team members through their own accounts. Add the real URL to README.

2. Each member configures their own git user.name and user.email. Do not create commits under another participant's identity.

3. Review and upload the package in an honestly dated initial commit acknowledging assistant support. Record subsequent real changes in small commits.

4. Create a YouTrack project and add members. Use State, Priority, Assignee, Due Date and Type. Suggested states are Open, In Progress, Review and Done, with Blocked separately. Do not mark unreviewed drafts Done.

5. Import the 22 prepared tasks or create them manually. Run scripts/import_youtrack.py without --apply first to inspect requests without sending them. Actual import requires the URL, project ID and token in environment variables. The script includes owners, dates and priorities in descriptions; set and verify the actual custom fields separately.

6. Name branches feat/<real-issue-id>-short-name. Link each PR to its task, acceptance criterion and verification. A different team member reviews the change. Protect main from direct pushes and require at least one approval.

7. Before submission, verify links, agreement and materials. Create the annotated pre-defense-1 tag on the reviewed commit. Later milestones use pre-defense-2 and final-pre-defense. Do not move submitted tags; use a new commit and an agreed additional tag for corrections.

First upload on Windows

Open PowerShell in the extracted ReproService folder. If it is not yet a repository, run git init -b main, then git status. Configure your own identity and review files. Run git add . and git commit -m "Prepare initial thesis proposal and review materials". Use the actual remote URL from your repository: git remote add origin <REAL_URL>, followed by git push -u origin main.

The YouTrack importer follows JetBrains API documentation [T09] and keeps created IDs. If a network error follows a request, check the server before retrying to prevent duplicate tasks. Do not store tokens in Git.

Check against the assigned topic list

The supplied Titles of Diploma Projects 2026-27 (for Students)(1).xlsx was checked on 14 October 2026. Fifty sheets and 291 numbered topic records were examined using normalized title comparison and Russian/English keyword searches. No exact match was found. This concerns the supplied version only and is neither official university approval nor proof of worldwide uniqueness.

Lecturer sheet and row	Nearby topic in English	Difference
Қожахмет Жақсылық Ерданұлы / 7	Distributed messaging and low-latency voice platform using event-driven microservices	Microservices are the architecture of a communication product; our product tests interservice failures.
Оспанов Алмас Даулетыярович / 7	Automatic test-case generation from requirements using a local language model	It generates tests from documents; our work controls HTTP interventions and reduces a failure scenario.
Өмірзақ Ислам Аманжолұлы / 6	Automated API contract testing with boundary payload generation from OpenAPI/Swagger	It checks contracts and generates inputs; our focus is retries, fault timing and business invariants.
Шутеева Гулнур Сериковна / 3	Dashboard for automated testing results	It displays results; our main contribution concerns execution control and reduction.

Lecturer names retain the spelling used in the source workbook. The school and supervisor decide whether the topic is acceptable and sufficiently distinct. Repeat the check if the list changes.

Evidence needed to finish the report

Action	What to provide	Target date
Conduct 3–5 interviews	Anonymized date, role, incident, tools, constraints and permission to use notes	18 October
Obtain supervisor feedback	Comments on title, scope, experiment and roles, plus the team's response to each	21 October
Set up Git and YouTrack	Working links, team access, 22 assigned tasks and authentic participant changes	15 October
Review as a team	Corrections to owned sections and confirmation of responsibilities	22 October
Complete submission checks	Verified links, current PDF and a presentation rehearsal	23 October

13 References

S identifies scientific publications, D product sources and T technical documentation. Electronic sources were accessed on 14 October 2026. The review depth is stated to distinguish full-text checks from abstract-level review. For continuously updated documentation, 2026 refers to the consultation year.

[S01] D. Flipse, H. Simsek, J. Decouchant, B. Kulahcioglu Ozkan Automated Network-Level Fault Injection Testing of Microservice Architectures. ICSE 2026, pp. 3886–3897. 2026.

https://research.tudelft.nl/en/publications/automated-network-level-fault-injection-testing-of-microservice-a/

Reviewed: Full-text Sections 3.1–3.5, 4 and 5.4; README

[S02] C. S. Meiklejohn, A. Estrada, Y. Song, H. Miller, R. Padhye Service-Level Fault Injection Testing. ACM Symposium on Cloud Computing. 2021.

https://christophermeiklejohn.com/publications/filibuster-socc-2021.pdf

Reviewed: Full-text Sections 3.1.2–3.1.4 and 4, Algorithm 1

[S03] X. Zhou, X. Peng, T. Xie, J. Sun, C. Ji, W. Li, D. Ding Delta debugging microservice systems with parallel optimization. IEEE Transactions on Services Computing 15(1), pp. 16–29. 2022.

https://ink.library.smu.edu.sg/sis_research/4844/

Reviewed: Abstract and university bibliographic record

[S04] W. Feng, Q. Pei, Y. Gao, D. Wang, W. Dou, J. Wei, Z. Liang, Z. Long FaultFuzz: A Coverage Guided Fault Injection Tool for Distributed Systems. ICSE Companion 2024. 2024.

https://wsdou.github.io/papers/2024-icse-faultfuzz.pdf

Reviewed: Open full text

[S05] J. Pan, H. Wu, T. Leesatapornwongsa, S. Nath, P. Huang Efficient Reproduction of Fault-Induced Failures in Distributed Systems with Feedback-Driven Fault Injection. ACM SOSP 2024. 2024.

https://www.microsoft.com/en-us/research/publication/feedback-driven-fault-injection-efficiently-reproducing-fault-induced-failures/

Reviewed: Author abstract and bibliographic record

[S06] H. Wu, J. Pan, P. Huang Efficient Exposure of Partial Failure Bugs in Distributed Systems with Inferred Abstract States. USENIX NSDI 2024. 2024.

https://www.usenix.org/conference/nsdi24/presentation/wu-haoze

Reviewed: Conference abstract

[S07] Y. Chen, F. Ma, Y. Zhou, Z. Yan, Y. Jiang CAFault: Enhance Fault Injection Technique in Practical Distributed Systems via Abundant Fault-Dependent Configurations. USENIX ATC 2025. 2025.

https://www.usenix.org/conference/atc25/presentation/chen-yuanliang

Reviewed: Conference abstract

[S08] M. Assad, C. S. Meiklejohn, H. Miller, S. Krusche Can My Microservice Tolerate an Unreliable Database? Resilience Testing with Fault Injection and Visualization. ICSE Companion 2024, pp. 54–58. 2024.

https://arxiv.org/abs/2404.01886

Reviewed: Author abstract; publication verified through TUM

[S09] C. Scott, A. Panda, V. Brajkovic, G. Necula, A. Krishnamurthy, S. Shenker Minimizing Faulty Executions of Distributed Systems. USENIX NSDI 2016, pp. 291–309. 2016.

https://www.usenix.org/conference/nsdi16/technical-sessions/presentation/scott

Reviewed: Full-text Sections 2.3, 3.2.1 and 4.1–4.2

[D01] Reynard contributors Reynard README. Official repository. 2026.

https://github.com/reynard-testing/reynard

Reviewed: README

[D02] Filibuster Filibuster. Official website. 2026.
https://www.filibuster.cloud/
Reviewed: Official website
[D03] Antithesis Antithesis platform. Official website. 2026.
https://antithesis.com/product/
Reviewed: Platform description
[D04] Chaos Mesh Chaos Mesh Overview. Official documentation. 2026.
https://chaos-mesh.org/docs/
Reviewed: Overview
[D05] Keploy Keploy documentation. Official documentation. 2026.
https://keploy.io/docs/
Reviewed: Overview
[T01] Docker Docker Compose. Official documentation. 2026.
https://docs.docker.com/compose/
Reviewed: Overview
[T02] The Go Authors Go documentation. Official documentation. 2026.
https://go.dev/doc/
Reviewed: Overview
[T03] Python Software Foundation asyncio. Official documentation. 2026.
https://docs.python.org/3/library/asyncio.html
Reviewed: Overview
[T04] PostgreSQL Global Development Group What is PostgreSQL. Official documentation. 2026.
https://www.postgresql.org/docs/current/intro-whatis.html
Reviewed: Overview
[T05] SQLite SQLite Is Serverless. Official documentation. 2026.
https://www.sqlite.org/serverless.html
Reviewed: Overview
[T06] W3C Trace Context. W3C Recommendation. 2021.
https://www.w3.org/TR/trace-context/
Reviewed: Specification
[T07] React Quick Start. Official documentation. 2026.
https://react.dev/learn
Reviewed: Overview
[T08] Vue.js Introduction. Official documentation. 2026.
https://vuejs.org/guide/introduction.html
Reviewed: Overview
[T09] JetBrains Create an Issue. YouTrack Developer Portal. 2026.
https://www.jetbrains.com/help/youtrack/devportal/api-howto-create-issue.html
Reviewed: REST API guide